Assignment 2 Final Report
Cell-DEVS Modeling and Simulation of a Cyclic Cellular Automaton

Maryam Bazyari

Abstract

This report presents the complete modeling, implementation, execution, and analysis of a cyclic cellular automaton using the Cell-DEVS formalism and the Cadmium simulation framework. The selected system is a two-dimensional discrete grid in which each cell stores one state from a cyclic set of values and updates its state according to local interactions with neighboring cells. A three-state model with states 0, 1, and 2 was implemented. The local rule causes a cell to advance to the next state in the cycle whenever that next state appears in the neighborhood. The model was built in Cadmium, configured through JSON scenario files, executed under multiple experimental conditions, and analyzed through generated log files and visualization outputs. Three experiments were performed to study the effect of initial distributions and boundary conditions. The results show that simple local cyclic interactions generate structured spatial propagation and evolving wave-like patterns over time.

Keywords: Cell-DEVS, Cadmium, cyclic cellular automaton, cellular automata, spatial simulation, discrete-event simulation

1 Introduction

Cellular automata are discrete dynamical systems composed of cells arranged in a regular lattice, where the evolution of each cell depends on its current state and the states of neighboring cells. These models are widely used to study local interaction rules, emergent behavior, and spatial pattern formation.

In this assignment, the selected system is a cyclic cellular automaton represented as a Cell-DEVS model. The system is defined on a two-dimensional grid, where each cell can take one of three values and evolve according to a cyclic replacement rule. The model is suitable for Cell-DEVS because it is spatial, discrete, and entirely governed by local neighbor-based state transitions.

The objective of this work is to provide a complete Cell-DEVS treatment of the selected topic, including conceptual modeling, formal specification, implementation in Cadmium, simulation under multiple experimental settings, graphical visualization of results, and preparation of all required submission materials.

2 Conceptual Description

The cyclic cellular automaton is modeled as a two-dimensional lattice of cells. Each cell stores one state from the set

$$S = \{0, 1, 2\}.$$

The state progression is cyclic:

$$0 \to 1 \to 2 \to 0.$$

The conceptual rule is simple. A cell changes to the next state in the cycle whenever that next state appears in its neighborhood. If the required neighbor state is not present, the cell keeps its current value. Therefore:

- a cell in state 0 changes to 1 if at least one neighbor is in state 1;
- a cell in state 1 changes to 2 if at least one neighbor is in state 2;
- a cell in state 2 changes to 0 if at least one neighbor is in state 0;
- otherwise, the cell remains unchanged.

This mechanism creates spatial propagation and repeated cyclic replacement across the grid. Although the local rule is simple, the repeated application of that rule over a cell space generates visible spatiotemporal behavior.

3 Formal Cell-DEVS Specification

The model is formally defined as a Cell-DEVS grid-based system with the following components.

3.1 Cell Space

The cell space is a two-dimensional finite lattice:

$$C \subseteq \mathbb{Z}^2.$$

For the experiments developed in this assignment, a 20×20 grid was used.

3.2 State Set

Each cell state belongs to:

$$S = \{0, 1, 2\}.$$

3.3 Neighborhood

A Moore neighborhood with range 1 is used. Therefore, each cell can observe the eight surrounding cells.

3.4 Local Transition Function

Let s denote the current state of a cell and let N denote its neighborhood. The local transition function is:

$$\tau(s, N) = \begin{cases} 1, & \text{if } s = 0 \text{ and } \exists n \in N \text{ such that } n = 1 \\ 2, & \text{if } s = 1 \text{ and } \exists n \in N \text{ such that } n = 2 \\ 0, & \text{if } s = 2 \text{ and } \exists n \in N \text{ such that } n = 0 \\ s, & \text{otherwise} \end{cases}$$

3.5 Delay

A fixed output delay of one time unit is assigned to all cells:

$$d = 1.$$

3.6 Boundary Conditions

Two boundary settings were studied:

- non-wrapped boundaries;
- wrapped boundaries.

4 Model Implementation in Cadmium

The system was implemented in Cadmium using a grid Cell-DEVS structure. The implementation includes the state definition, the atomic cell behavior, the main simulation program, and JSON configuration files.

4.1 State File

The file `include/cyclic_state.hpp` defines the cell state structure. The state contains a single integer variable named `phase`, which stores the value 0, 1, or 2. The file also includes:

- a default constructor;
- a comparison operator for state change detection;
- an output operator for logging;
- JSON parsing support.

4.2 Atomic Grid Cell

The file `include/cyclic_grid_cell.hpp` implements the atomic cell model. The local transition function reads the states of neighboring cells and applies the cyclic rule:

- if the current state is 0 and a neighbor with state 1 exists, the cell becomes 1;
- if the current state is 1 and a neighbor with state 2 exists, the cell becomes 2;
- if the current state is 2 and a neighbor with state 0 exists, the cell becomes 0;
- otherwise, the state is unchanged.

The same file defines a fixed output delay of 1.0 for all cells.

4.3 Main Simulation Program

The file `main_cyclic.cpp` performs the following tasks:

1. reads the configuration file path from the command line;
2. loads the Cell-DEVS grid model;
3. builds the model;
4. creates the root coordinator;
5. assigns a CSV logger;
6. starts simulation;
7. executes the simulation for a selected duration;
8. stops simulation.

4.4 Configuration Files

Three JSON configuration files were created:

- `cyclic_config.json`
- `cyclic_config_wrapped.json`
- `cyclic_config_stripes.json`

These files define the scenario shape, origin, boundary setting, default cell state, neighborhood type, and custom initial patterns.

5 Experimental Frame

To analyze the behavior of the model, three experiments were defined.

5.1 Experiment 1: Basic Configuration

- Grid size: 20×20
- Boundary condition: non-wrapped
- Neighborhood: Moore, range 1
- Initial state:
  - most cells initialized to state 0;
  - a central patch initialized to state 1;
  - two additional patches initialized to state 2.

  This was used as the reference experiment.

5.2 Experiment 2: Wrapped Boundary Configuration

- Grid size: 20×20
- Boundary condition: wrapped
- Neighborhood: Moore, range 1
- Initial state:
  - corner regions initialized to state 1;
  - a central patch initialized to state 2;
  - all remaining cells initialized to state 0.

  This experiment was designed to evaluate the effect of periodic boundaries.

5.3 Experiment 3: Stripe Initialization Configuration

- Grid size: 20×20
- Boundary condition: non-wrapped
- Neighborhood: Moore, range 1
- Initial state:
  - one stripe initialized to state 1;
  - one stripe initialized to state 2;
  - all remaining cells initialized to state 0.

  This experiment was designed to generate directional state propagation fronts.

5.4 Simulation Duration

Each experiment was simulated for 50 time units.

6 Execution and Output Files

The model was compiled and executed successfully. During execution, Cadmium generated CSV log files describing the state of individual cells over time.

The following output files were produced:

- `logs/cyclic_basic_log.csv`
- `logs/cyclic_wrapped_log.csv`
- `logs/cyclic_stripes_log.csv`

The generated output contains time values, cell identifiers, model names, and logged state data. These files were then used to generate visual outputs.

7 Visualization of Results

The generated simulation logs were post-processed to create graphical visualizations of the model behavior. Three representative snapshots were generated for the basic experiment:

- initial state,
- intermediate state,
- final state.

An animation file was also created from the simulation log to show the evolution of cell states over time.

The generated visualization files are:

- `visualization_output/start.png`
- `visualization_output/middle.png`
- `visualization_output/end.png`
- `visualization_output/cyclic_animation.gif`

7.1 Initial State

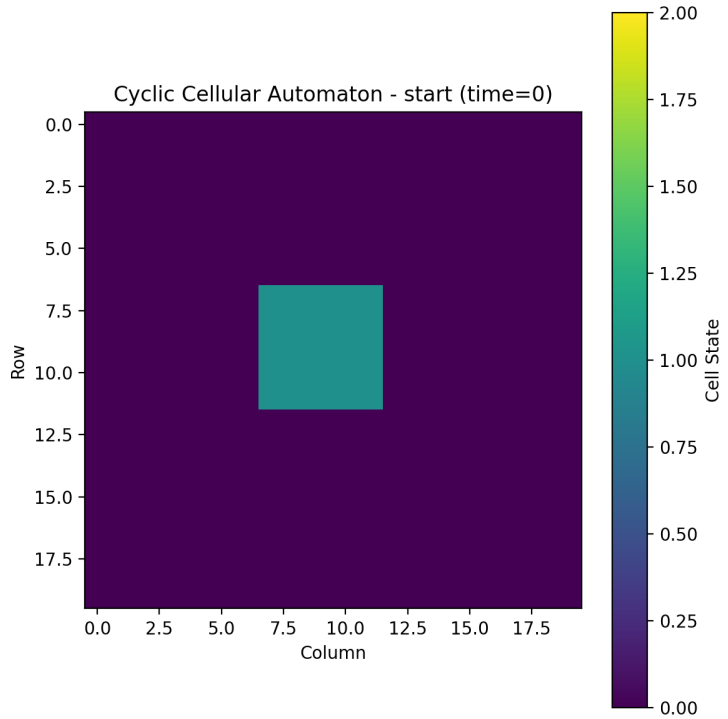

Figure 1: Initial configuration of the cyclic cellular automaton.

7.2 Intermediate State

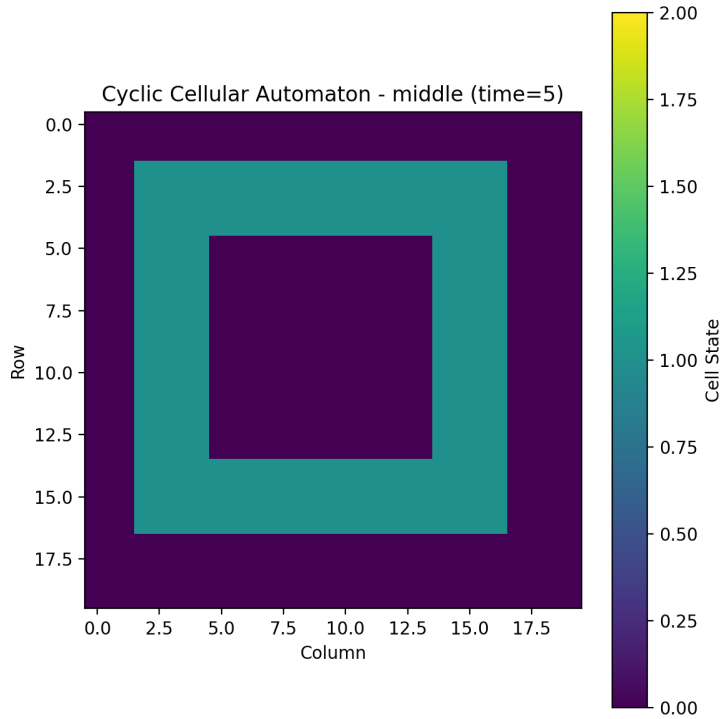

Figure 2: Intermediate state showing propagation of cyclic transitions across the grid.

7.3 Final State

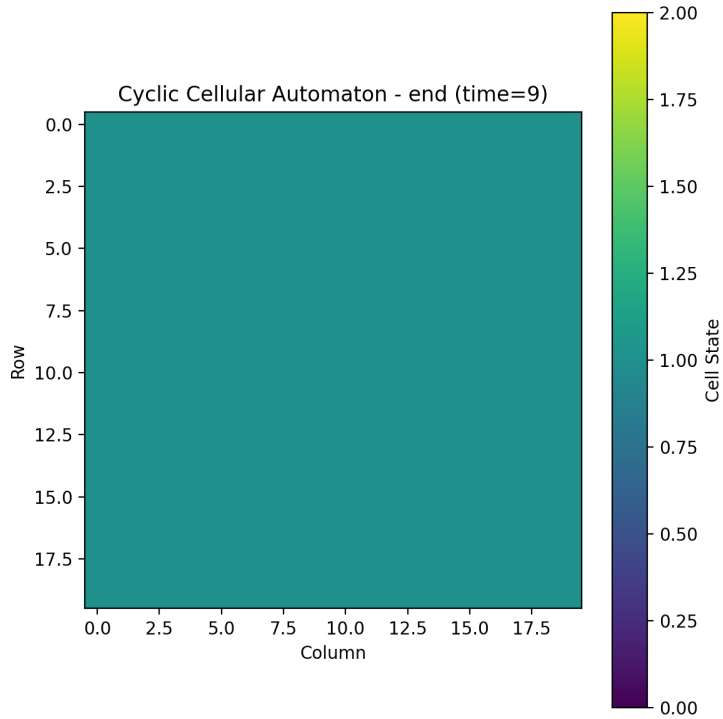

Figure 3: Final state after the selected simulation duration.

The visualized propagation does not form a perfectly smooth geometric circle. Instead, the wavefront appears square-like or lattice-shaped. This is expected because the model is implemented on a discrete two-dimensional square grid and uses local neighborhood interactions. The geometry of the lattice therefore influences the visual form of propagation.

Even with that discrete appearance, the generated outputs clearly show cyclic propagation, state replacement, and spatial evolution over time.

8 Analysis of Results

The results demonstrate that the cyclic rule successfully generates spatially distributed behavior.

8.1 Basic Configuration

In the basic configuration, the initial state patches triggered outward propagation. The central state-1 region influenced neighboring state-0 cells, while state-2 regions introduced additional cyclic change. The model produced visible wave-like evolution over the lattice.

8.2 Wrapped Boundary Configuration

When wrapped boundaries were used, cells at the edges interacted with cells on the opposite side of the grid. This created more continuous propagation across the border and reduced edge isolation effects.

8.3 Stripe Configuration

The stripe-based initialization produced more directional patterns. Since large groups of cells in state 1 and state 2 were aligned along narrow regions, the resulting evolution emphasized linear propagation and stronger front-like transitions.

8.4 General Observations

Across all experiments, the following behaviors were observed:

- local state rules generated nontrivial global spatial patterns;
- initial distribution strongly influenced the resulting behavior;
- boundary conditions affected propagation near the edges;
- the model remained consistent with the intended cyclic replacement logic.

9 Repository and Submission Materials

All required project materials were organized in the repository, including implementation files, configuration files, generated logs, visualization outputs, and documentation.

The repository contents include:

- `main_cyclic.cpp`
- `include/cyclic_state.hpp`
- `include/cyclic_grid_cell.hpp`
- `CMakeLists.txt`
- `cyclic_config.json`
- `cyclic_config_wrapped.json`
- `cyclic_config_stripes.json`
- `logs/` folder
- `visualization_output/` folder
- `videos/` folder

- `README.md`
- final report
- DEVSmodelsForm

The README file contains build and execution instructions. External dependencies such as Cadmium, Boost, and JSON headers are listed as required tools.

10 Discussion

The cyclic cellular automaton proved to be a suitable example of a Cell-DEVS system because its behavior is entirely determined by local, discrete, neighbor-based interactions. The implementation in Cadmium matched the intended conceptual structure and allowed the model to be simulated under different settings.

A limitation of the present implementation is the relatively small grid size. A larger grid could produce more visually refined propagation patterns. Another limitation is that only three states were used. More complex variants could use more states or more elaborate transition rules. However, the three-state system is sufficient to demonstrate the intended cyclic mechanism and its spatial consequences.

The visualization stage confirmed that simulation logs can be used effectively to illustrate Cell-DEVS behavior. The generated images and animation clearly show the time evolution of the system.

11 Conclusion

This work presented a complete Cell-DEVS modeling and simulation study of a cyclic cellular automaton using Cadmium. The system was defined on a two-dimensional grid with three possible states and a cyclic neighbor-based transition rule. The model was formally specified, implemented in Cadmium, configured using JSON files, simulated under multiple conditions, and analyzed using generated log files and visualization outputs.

The results demonstrate that simple cyclic local interaction rules are sufficient to create evolving spatial structures and wave-like behavior. Different initial conditions and boundary assumptions produced different visual outcomes while preserving the same core cyclic dynamics. The project successfully fulfills the objectives of conceptual modeling, formal specification, implementation, simulation, visualization, and result analysis required for the assignment.

References

[1] R. Fisch, J. Gravner, and D. Griffeath, "Cyclic cellular automata in two dimensions," in *Spatial Stochastic Processes: A Festschrift in Honor of Ted Harris on his Seventieth Birthday*, Boston, MA: Birkhäuser, 1991, pp. 171–185.